

namespaces

Types of Namespaces in Linux

1. Mount Namespace (`mnt`)

- **Isolates file system mount points.**
- Allows processes to have **separate views of the filesystem hierarchy**.
- Example: Containers can mount their own filesystems without affecting the host system.

```
unshare -m /bin/bash
```

2. PID Namespace (`pid`)

- **Isolates process IDs.**
- Processes in different PID namespaces can have **the same process ID (PID)**, but they are **independent** of each other.
- Example: Containers run processes that appear to start with PID **1**, mimicking a new OS instance.

Command to create:

```
unshare -p --fork /bin/bash
```

3. Network Namespace (`net`)

- **Isolates network resources** (interfaces, routing tables, firewall rules).
- Each namespace can have its **own IP addresses, ports, and routing rules**.
- Example: Containers can run services bound to **localhost** without conflicts.

Command to create:

```
unshare -n /bin/bash
```

4. IPC Namespace (`ipc`)

- **Isolates inter-process communication (IPC)** resources such as **shared memory** and **message queues**.
- Processes in different IPC namespaces cannot **communicate with each other** using IPC mechanisms.

Command to create:

```
unshare -i /bin/bash
```

5. UTS Namespace (**uts**)

- **Isolates hostname and domain name.**
- Processes in different UTS namespaces can have **different hostnames**.
- Example: Containers can have their **own hostname**, different from the host.

Command to create:

```
unshare -u /bin/bash
hostname container-host
```

6. User Namespace (**user**)

- **Isolates user and group IDs (UID and GID).**
- Allows processes to have **different user IDs** inside and outside the namespace.
- Example: Containers can run as **root** inside the namespace but map to **non-root users** on the host.

Command to create:

```
unshare -U /bin/bash
```

7. Cgroup Namespace (**cgroup**)

- **Isolates view of cgroups** (control groups), which are used to **limit resources** like CPU and memory.
- Example: Processes can have **different resource quotas** without affecting others.

Each NS isolates some kind of resource(s)

- Each NS type isolates some kind of resource(s):
 - **UTS** NSs: isolate system identifiers (e.g., hostname)
 - **Mount** NSs: isolate mount point list
 - **IPC** NSs: isolate interprocess communication resources
 - **PID** NSs: isolate PID number space
 - **Network** NSs: isolate NW resources
 - Firewall & routing rules, socket port numbers, `/proc/net`, `/sys/class/net`, ...
 - And so on....

UTS namespaces

- UTS NSs are simple, and so provide an easy example
- Isolate two system identifiers returned by `uname(2)`
 - *nodename*: system hostname (set by `sethostname(2)`)
 - *domainname*: NIS domain name (set by `setdomainname(2)`)
- Container configuration scripts might tailor their actions based on these IDs
 - E.g., nodename could be used with DHCP, to obtain IP address for container
- “UTS” comes from *struct utsname* argument of `uname(2)`
 - Structure name derives from “UNIX Timesharing System”

Background: capabilities

- Capabilities: **divide power of superuser into small pieces**
 - 41 capabilities as at Linux 6.4 (see [capabilities\(7\)](#))
- Examples:
 - `CAP_DAC_OVERRIDE`: bypass all file permission checks
 - `CAP_SYS_ADMIN`: do (too) many different sysadmin tasks
 - `CAP_SYS_TIME`: change system time
- Instead of set-UID-*root* programs, have programs with one/a few attached capabilities
 - Attached using [setcap\(8\)](#)
 - When program is executed $\Rightarrow$ process gets those capabilities
- Program is **weaker** than set-UID-*root* program
 - $\Rightarrow$ **less dangerous if compromised**



UID and GID mappings

- One of first steps after creating a user NS is to define **UID and GID mappings** for NS
 - Defined by writing to 2 files: `/proc/PID/uid_map` and `/proc/PID/gid_map`
- Records written to/read from `uid_map` and `gid_map` have the form:

ID-inside-ns	ID-outside-ns	length
--------------	---------------	--------

- *ID-inside-ns* and *length* define range of IDs inside user NS that are to be mapped
 - *ID-outside-ns* defines start of corresponding mapped range in "outside" user NS
- Commonly these files are initialized with a single line containing "root mapping":

0	1000	1
---	------	---

- I.e., UID 0 inside NS maps to unprivileged UID in outer NS